1. Basic Kafka Operations - Command-line Tools - Partially 2
2. Writing a Kafka Producer in Java 12
3. Consumer – Java API 26
4. Java API – Producer – II 36
5. Java API – Consumer – II 58

1. Basic Kafka Operations - Command-line Tools - Partially

You need to start the broker using startABroker.sh. The script should be in /opt/scripts folder
#sh startABroker.sh
#jps

```
[root@tos scripts]# sh startABroker.sh
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
Started Successfully
[root@tos scripts]# jps
11665 Jps
11646 Kafka
11375 QuorumPeerMain
[root@tos scripts]#
```

Once the Kafka broker is started, we can verify that it is working by performing some simple operations against the broker; creating a test topic, producing some messages, and consuming the same messages. Create and verify a details on topic:

/opt/kafka/bin/kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1 --topic test
/opt/kafka/bin/kafka-topics.sh --zookeeper localhost:2181 --describe --topic test

```
[root@tos opt]# /opt/kafka/bin/kafka-topics.sh --create --zookeeper localhost:21
81 --replication-factor 1 --partitions 1 --topic test
Created topic "test".
[root@tos opt]# /opt/kafka/bin/kafka-topics.sh --zookeeper localhost:2181 --desc
ribe --topic test
Topic:test      PartitionCount:1        ReplicationFactor:1     Configs:
        Topic: test     Partition: 0    Leader: 0       Replicas: 0     Isr: 0
[root@tos opt]#
```

Produce messages to a test topic: It will open a console to send message to the topic, test. Enter some text as shown below.

/opt/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092 --topic test
Test Message 1
Test Message 2
^D
#

```
[root@tos config]# /opt/kafka/bin/kafka-console-producer.sh --broker-list localh
ost:9092 --topic test
>hi
>Hello
>TEst message
>[root@tos config]#
```

Consume messages from a test topic: As soon as you enter the following script in a separate terminal, You should be able to consume the messages that we have type in the producer console.

/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test --from-beginning

```
[root@tos config]# /opt/kafka/bin/kafka-console-consumer.sh -zookeeper localhost
:2181 --topic test --from-beginning
Using the ConsoleConsumer with old consumer is deprecated and will be removed in
 a future major release. Consider using the new consumer by passing [bootstrap-s
erver] instead of [zookeeper].
hi
Hello
TEst message
^CProcessed a total of 3 messages
```

Create, list and describe topics.

#/opt/kafka/bin/kafka-topics.sh --zookeeper localhost:2181 -list

```
[root@tos ~]# /opt/kafka/bin/kafka-topics.sh --zookeeper localhost:2181 --list
__consumer_offsets
my-failsafe-topic
test
```

List and describe Topics

What does the tool do?

This tool lists the information for a given list of topics. If no topics are provided in the command line, the tool queries zookeeper to get all the topics and lists the information for them. The fields that the tool displays are - topic name, partition, leader, replicas, isr. Two optional arguments can be provided to the tool. If "under-replicated-partitions" is specified, the tool only provides information for those topic / partitions which have replicas that are under replicated. If "unavailable-partitions" is specified, the tool only provides information for those topic/partitions whose leader is not available.

How to use the tool?

List only single topic named "test" (prints only topic name)

/opt/kafka/bin/kafka-topics.sh --list --zookeeper localhost:2181 --topic test

List all topics (prints only topic names)

#/opt/kafka/bin/kafka-topics.sh --list --zookeeper localhost:2181

```
[root@tos scripts]# jps
12960 Jps
12314 Kafka
12043 QuorumPeerMain
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --list --zookeeper tos.master.com:2181 --topic
 test
test
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --list --zookeeper localhost:2181
CustomerCountry
__consumer_offsets
henry-topic
my-failsafe-topic
my-kafka-topic
my-kafka-topic1
test
test-topic
```

Describe only single topic named "test" (prints details about the topic)
#/opt/kafka/bin/kafka-topics.sh --describe --zookeeper localhost:2181 --topic test

Describe all topics (prints details about the topics)
#/opt/kafka/bin/kafka-topics.sh --describe --zookeeper localhost:2181

```
[root@tos scripts]#
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --describe --zookeeper localhost:2181 --topic
test
Topic:test      PartitionCount:1        ReplicationFactor:1     Configs:
        Topic: test     Partition: 0    Leader: 0       Replicas: 0     Isr: 0
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --describe --zookeeper localhost:2181
Topic:CustomerCountry   PartitionCount:1        ReplicationFactor:1     Configs:
        Topic: CustomerCountry  Partition: 0    Leader: 1       Replicas: 1     Isr: 1
Topic:__consumer_offsets        PartitionCount:50       ReplicationFactor:1     Configs:segment.b
ytes=104857600,cleanup.policy=compact,compression.type=producer
        Topic: __consumer_offsets       Partition: 0    Leader: 1       Replicas: 1     Isr: 1
        Topic: __consumer_offsets       Partition: 1    Leader: 2       Replicas: 2     Isr: 2
```

We will understand in details later in the tutorials.
Create Topics
What does the tool do?
By default, Kafka auto creates topic if "auto.create.topics.enable" is set to true on the server. This creates a topic with a default number of partitions, replication factor and uses Kafka's default scheme to do replica assignment. Sometimes, it may be required that we would like to customize a topic while creating it. This tool helps to create a topic and also specify the number of partitions, replication factor and replica assignment list for the topic.
How to use the tool?
create topic with default settings
/opt/kafka/bin/kafka-topics.sh --create --zookeeper localhost:2181 --topic topic1 --partitions 2 --replication-factor 1

```
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --create --zookeeper localhost:2181 --topic to
pic1 --partitions 2 --replication-factor 1
Created topic "topic1".
[root@tos scripts]#
```

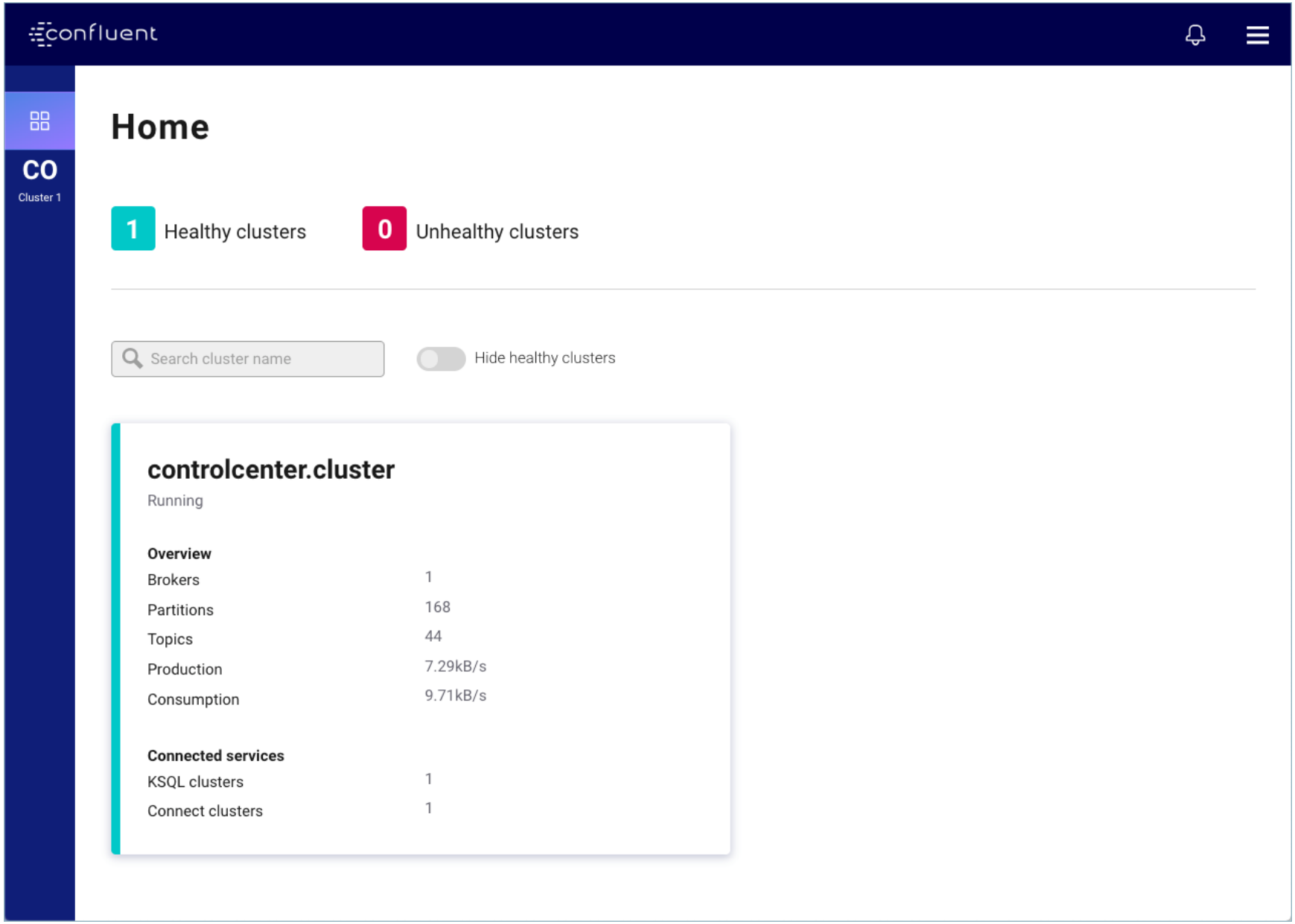
Create Kafka Topics

In this step, you create Kafka topics by using the Confluent Control Center. Confluent Control Center provides the functionality for building and monitoring production data pipelines and event streaming applications.

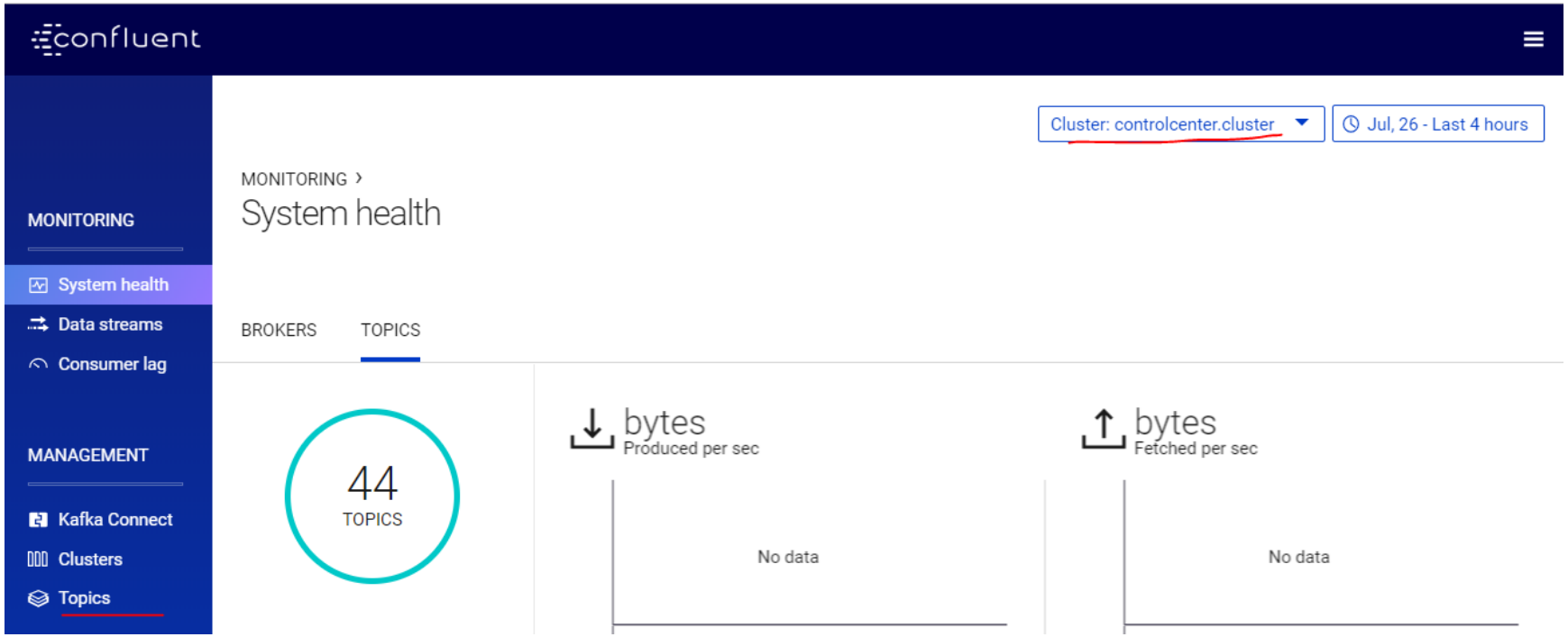
1. Navigate to the Control Center web interface at http://localhost:9021/.

Important

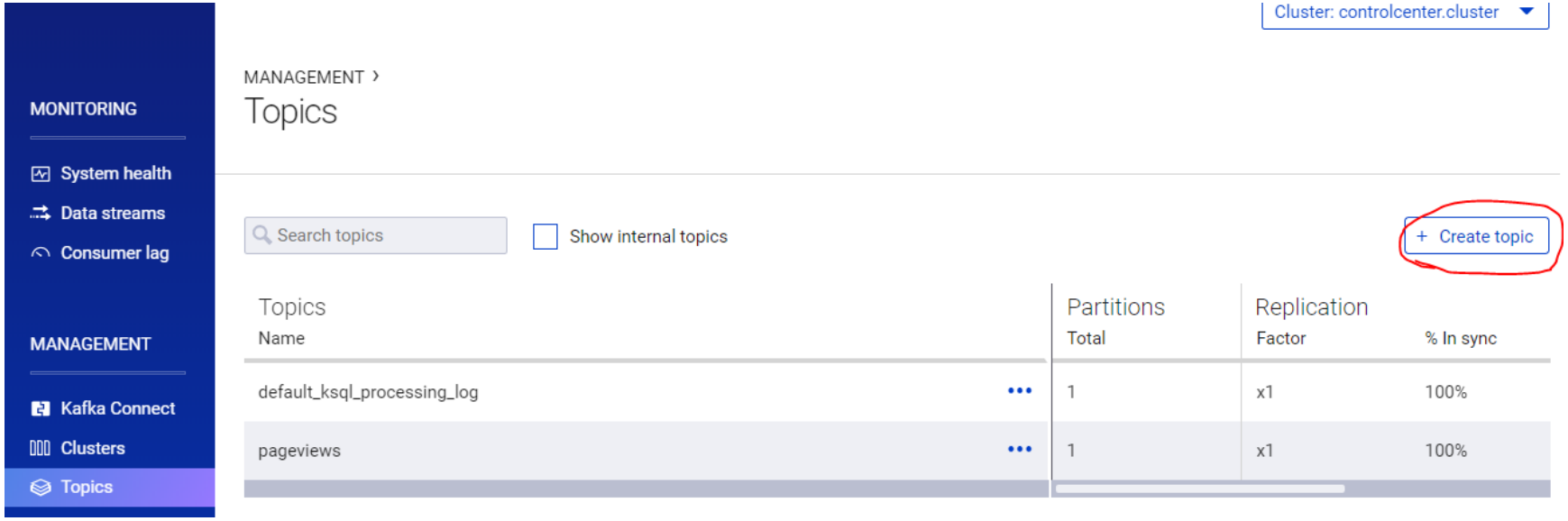
It may take a minute or two for Control Center to come online.


confluent
CO
Cluster 1
Home
1 Healthy clusters
0 Unhealthy clusters
Search cluster name
Hide healthy clusters
controlcenter.cluster
Running
Overview
Brokers 1
Partitions 168
Topics 44
Production 7.29kB/s
Consumption 9.71kB/s
Connected services
KSQL clusters 1
Connect clusters 1

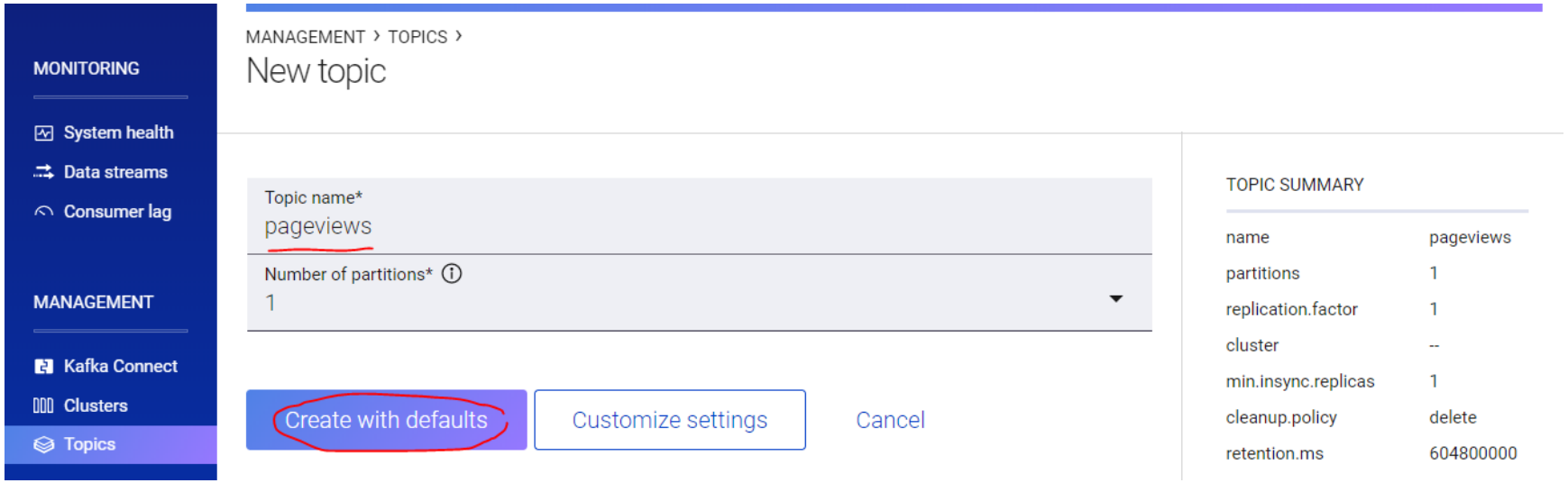
2. Select your cluster name.



3. Select **Topics** from the cluster submenu and click **Create topic**.



4. Create a topic named pageviews and click **Create with defaults**.



5. Repeat the previous steps and create a topic named users and click **Create with defaults**.

Lab CLI completes End here.

2. Writing a Kafka Producer in Java

In this tutorial, we are going to create a simple Java example that creates a Kafka producer. You need to create a new replicated Kafka topic called my-kafka-topic, then you will create a Kafka producer using Java API that uses this topic to send records. You will send records with the Kafka producer. You will send records synchronously.
You need to start the zookeeper and three nodes brokers before going ahead.

#cd /opt/scripts
#sh start3Brokers.sh

```
[root@tos scripts]# jps
4880 Kafka
4881 Kafka
4882 Kafka
6022 Jps
4845 QuorumPeerMain
[root@tos scripts]#
```

Here, as shown above three Kafka broker services and ZK service need to be started.

Create Replicated Kafka Topic
Create topic
#/opt/kafka/bin/kafka-topics.sh --create --replication-factor 3 --partitions 13 --topic my-kafka-topic --zookeeper localhost:2181

Above we created a topic named my-kafka-topic with 13 partitions and a replication factor of 3. Then we list the Kafka topics.

```
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --create     --replication-fa
ctor 3    --partitions 13    --topic my-kafka-topic     --zookeeper  localhost:2
181
Created topic "my-kafka-topic".
[root@tos scripts]#
```

List created topics, you can verify the topic now
/opt/kafka/bin/kafka-topics.sh --list --zookeeper localhost:2181

```
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --list    --zookeeper localhost:2181
__consumer_offsets
my-failsafe-topic
my-kafka-topic
test
[root@tos scripts]#
```

We will use maven to create the java project. (You can refer the Annexure I – How to create Maven project).

Maven java project Details:
Group ID: com.tos.kafka
Artifact ID: my-kafka-producer.

After you create a maven java project, include the following dependency.

```
<dependencies>
          <dependency>
               <groupId>org.apache.kafka</groupId>
               <artifactId>kafka-clients</artifactId>
               <version>2.6.0</version>
          </dependency>
          <dependency>
               <groupId>org.apache.kafka</groupId>
               <artifactId>kafka-streams</artifactId>
               <version>2.6.0</version>
          </dependency>
          <dependency>
               <groupId>org.apache.kafka</groupId>
               <artifactId>kafka-streams-scala_2.13</artifactId>
               <version>2.6.0</version>
          </dependency>
   <dependency>
               <groupId>io.advantageous.boon</groupId>
               <artifactId>boon-json</artifactId>
               <version>0.6.6</version>
          </dependency>

          <dependency>
               <groupId>org.slf4j</groupId>
               <artifactId>slf4j-ext</artifactId>
               <version>1.7.13</version>
          </dependency>

          </dependencies>
   <build>
          <plugins>
               <plugin>
```

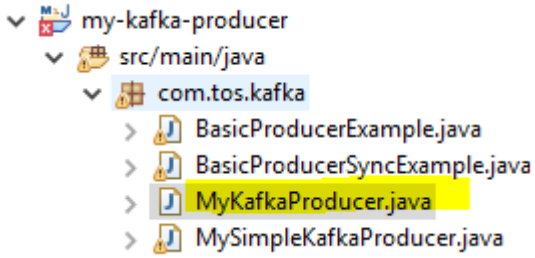
```
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.6.1</version>
                <configuration>
                    <source>1.8</source>
                    <target>1.8</target>
                </configuration>
            </plugin>

        </plugins>


</build>
```

Construct a Kafka Producer
To create a Kafka producer, you will need to pass it a list of bootstrap servers (a list of Kafka brokers). You will also specify a client.id that uniquely identifies this Producer client. In this example, we are going to send messages with ids. The message body is a string, so we need a record value serializer as we will send the message body in the Kafka's records value field. The message id (long), will be sent as the Kafka's records key. You will need to specify a Key serializer and a value serializer, which Kafka will use to encode the message id as a Kafka record key, and the message body as the Kafka record value.

Create a java class with the following package name:

Class : MyKafkaProducer
Package Name : com.tos.kafka

At the end of this lab, you will have a project structure as shown below:



Next, we will import the Kafka packages and define a constant for the topic and a constant to define the list of bootstrap servers that the producer will connect.

Add the following imports in your code.

```
import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.LongSerializer;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;
```

Notice that we have imports LongSerializer which gets configured as the Kafka record key serializer, and imports StringSerializer which gets configured as the record value serializer.

Then, let us define some constant variables as stated below,
BOOTSTRAP_SERVERS is set to tos.master.com:9092, tos.master.com:9093, tos.master.com:9094 which is the three Kafka servers that we started up in the last lesson. Go ahead and make sure all three Kafka servers are running.

The constant TOPIC is set to the replicated Kafka topic that we just created.

Add the following variables in your code.

```
private final static String TOPIC = "my-kafka-topic";
  private final static String BOOTSTRAP_SERVERS =
        "tos.master.com:9092,tos.master.com:9093,tos.master.com:9094";
```

Create Kafka Producer to send records
Now, that we imported the Kafka classes and defined some constants, let's create a Kafka producer.
Add the following function in the code.

```
private static Producer<Long, String> createProducer() {
      Properties props = new Properties();

props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,BOOTSTRAP_SERV
ERS);
      props.put(ProducerConfig.CLIENT_ID_CONFIG, "KafkaExampleProducer");

props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,LongSerializer.class.
getName());
      props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
                                StringSerializer.class.getName());
      return new KafkaProducer<>(props);
   }
```

To create a Kafka producer, you use **java.util.Properties** and define certain properties that we pass to the constructor of a **KafkaProducer**.

Above **createProducer** sets the **BOOTSTRAP_SERVERS_CONFIG** ("bootstrap.servers) property to the list of broker addresses we defined earlier. **BOOTSTRAP_SERVERS_CONFIG** value is a comma separated list of host/port pairs

that the Producer uses to establish an initial connection to the Kafka cluster. The producer uses of all servers in the cluster no matter which ones we list here. This list only specifies the initial Kafka brokers used to discover the full set of servers of the Kafka cluster. If a server in this list is down, the producer will just go to the next broker in the list to discover the full topology of the Kafka cluster.

The CLIENT_ID_CONFIG ("client.id") is an id to pass to the server when making requests so the server can track the source of requests beyond just IP/port by passing a producer name for things like server-side request logging.

The **KEY_SERIALIZER_CLASS_CONFIG** ("key.serializer") is a Kafka **Serializer** class for Kafka record keys that implements the Kafka Serializer interface. Notice that we set this to **LongSerializer** as the message ids in our example are longs.

The **VALUE_SERIALIZER_CLASS_CONFIG** ("value.serializer") is a Kafka **Serializer** class for Kafka record values that implements the Kafka **Serializer** interface. Notice that we set this to **StringSerializer** as the message body in our example are strings.

Send records synchronously with Kafka Producer
Kafka provides a synchronous send method to send a record to a topic. Let's use this method to send some message ids and messages to the Kafka topic we created earlier.

Add the following in your code.

```
static void runProducer(final int sendMessageCount) throws Exception {
        final Producer<Long, String> producer = createProducer();
        long time = System.currentTimeMillis();

        try {
            for (long index = time; index < time + sendMessageCount; index++) {
                final ProducerRecord<Long, String> record = new
ProducerRecord<>(TOPIC, index, "Hello Kafka " + index);

                RecordMetadata metadata = producer.send(record).get();

            long elapsedTime = System.currentTimeMillis() - time;
                System.out.printf("Sent record(key=%s value=%s) " +
"meta(partition=%d, offset=%d) time=%d\n",
                        record.key(), record.value(), metadata.partition(),
metadata.offset(), elapsedTime);


            }
        } catch (Exception e) {

            e.printStackTrace();
```

```
        } finally {
            producer.flush();
            producer.close();
        }
    }
```

The above just iterates through a for loop, creating a ProducerRecord sending an example message ("Hello Kafka " + index) as the record value and the for loop index as the record key. For each iteration, runProducer calls the send method of the producer (RecordMetadata metadata = producer.send(record).get()). The send method returns a Java Future.

The response RecordMetadata has 'partition' where the record was written and the 'offset' of the record in that partition.

Notice the call to flush and close. Kafka will auto flush on its own, but you can also call flush explicitly which will send the accumulated records now. It is polite to close the connection when we are done.

Running the Kafka Producer.

Next you define the main method.

Add the following imports in your code.

```
public static void main(String[] args) {

        try {
            if (args.length == 0) {
                runProducer(5);
            } else {
                runProducer(Integer.parseInt(args[0]));
            }
        } catch (Exception e) {
            // TODO: handle exception
        }

    }
```

The main method just calls runProducer.

Start a consumer console so that you can consume the message sent by this producer.
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server tos.master.com:9094,localhost:9092 --topic my-kafka-topic --from-beginning

```
[root@tos scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
tos.master.com:9094,tos.master.com:9092 --topic my-kafka-topic   --from-beginnin
g
```

Execute the main program.

MyKafkaProducer.java

```java
import org.apache.kafka.common.serialization.LongSerializer;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;

public class MyKafkaProducer {

    private final static String TOPIC = "my-kafka-topic";
    private final static String BOOTSTRAP_SERVERS = "10.10.20.24:9092,10.10.20.24:9094";

    public static void main(String[] args) {

        try {
            if (args.length == 0) {
                runProducer(5);
            } else {
                runProducer(Integer.parseInt(args[0]));
            }
        } catch (Exception e) {
            // TODO: handle exception
        }

```

Problems | Target Platform State | Console

```
<terminated> MyKafkaProducer [Java Application] D:\MyExperiment\Java\jdk1.8.0_131\bin\javaw.exe (Jun 16, 2018, 5:11:08 PM)
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Picked up _JAVA_OPTIONS: -Djava.net.preferIPv4Stack=true
Sent record(key=1529149269560 value=Hello Kafka 1529149269560) meta(partition=12, offset=0) time=361
Sent record(key=1529149269561 value=Hello Kafka 1529149269561) meta(partition=6, offset=0) time=380
Sent record(key=1529149269562 value=Hello Kafka 1529149269562) meta(partition=6, offset=1) time=393
Sent record(key=1529149269563 value=Hello Kafka 1529149269563) meta(partition=9, offset=2) time=413
Sent record(key=1529149269564 value=Hello Kafka 1529149269564) meta(partition=2, offset=1) time=458
```

You should be able to view the message as shown above.
You can verify from the consumer console that whatever messages that were sent from the producer was consumed in the consumer console as shown below. In the next lab we will consume this from a Java client

```
[root@tos scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
tos.master.com:9094,tos.master.com:9092 --topic my-kafka-topic   --from-beginnin
g
How are you
Hope it works
hello
how
France
Hello Kafka 1529148792457
Hello Kafka 1529148792458
Hello Kafka 1529148792459
Hello Kafka 1529148792460
Hello Kafka 1529148792461
Hello Kafka 1529149269560
Hello Kafka 1529149269561
Hello Kafka 1529149269562
Hello Kafka 1529149269563
Hello Kafka 1529149269564
```

Conclusion Kafka Producer example
We created a simple example that creates a Kafka Producer. First, we created a new replicated Kafka topic; then we created Kafka Producer in Java that uses the Kafka

replicated topic to send records. We sent records with the Kafka Producer using sync send method.

--- --- ---

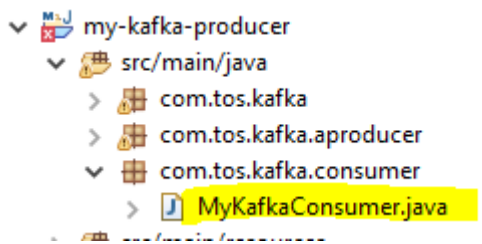
Lab End here

3. Consumer – Java API

In this tutorial, you are going to create simple *Kafka Consumer*. This consumer consumes messages from the Kafka Producer you wrote in the last tutorial. This tutorial demonstrates how to process records from a *Kafka topic* with a *Kafka Consumer*.
This tutorial describes how *Kafka Consumers* in the same group divide up and share partitions while each *consumer group* appears to get its own copy of the same data.
In the last tutorial, we created simple Java example that creates a Kafka producer. We also created replicated Kafka topic called my-example-topic, then you used the Kafka producer to send records (synchronously). Now, the consumer you create will consume those messages.
Construct a Kafka Consumer
Just like we did with the producer, you need to specify bootstrap servers. You also need to define a group.id that identifies which consumer group this consumer belongs. Then you need to designate a Kafka record key deserializer and a record value deserializer. Then you need to subscribe the consumer to the topic you created in the producer tutorial.
Kafka Consumer imports and constants

Next, you import the Kafka packages and define a constant for the topic and a constant to set the list of bootstrap servers that the consumer will connect.

KafkaConsumerExample.java - imports and constants
You can use the earlier java project. In that create a separate package and create the following class and package.
Package name : com.tos.kafka.consumer
MyKafkaConsumer.java




```
package com.tos.kafka.consumer;
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.clients.consumer.Consumer;
import org.apache.kafka.common.serialization.LongDeserializer;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.util.Collections;
import java.util.Properties;

public class MyKafkaConsumer {
   private final static String TOPIC = "my-kafka-topic";
   private final static String BOOTSTRAP_SERVERS =
         "10.10.20.24:9092,10.10.20.24:9093,10.10.20.24:9094";

}
```

Notice that KafkaConsumerExample imports LongDeserializer which gets configured as the Kafka record key deserializer, and imports StringDeserializer which gets set up as the record value

deserializer. The constant BOOTSTRAP_SERVERS gets set to localhost:9092,localhost:9093,localhost:9094 which is the three Kafka servers that we started up in the last lesson. Go ahead and make sure all three Kafka servers are running. The constant TOPIC gets set to the replicated Kafka topic that you created in the last tutorial.

Create Kafka Consumer consuming Topic to Receive Records
Now, that you imported the Kafka classes and defined some constants, let's create the Kafka consumer.
KafkaConsumerExample.java - Create Consumer to process Records
Add the following method that will initialize the consumer parameters.

```
 private static Consumer<Long, String> createConsumer() {
            final Properties props = new Properties();
            props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
BOOTSTRAP_SERVERS);
            props.put(ConsumerConfig.GROUP_ID_CONFIG, "KafkaExampleConsumer");
            props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
LongDeserializer.class.getName());
            props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());

            // Create the consumer using props.
            final Consumer<Long, String> consumer = new KafkaConsumer<>(props);

            // Subscribe to the topic.
            consumer.subscribe(Collections.singletonList(TOPIC));
            return consumer;
}
```

To create a Kafka consumer, you use java.util.Properties and define certain properties that we pass to the constructor of a KafkaConsumer.
Above KafkaConsumerExample.createConsumer sets
the BOOTSTRAP_SERVERS_CONFIG ("bootstrap.servers") property to the list of broker addresses we defined earlier. BOOTSTRAP_SERVERS_CONFIG value is a comma separated list of host/port pairs that the Consumer uses to establish an initial connection to the Kafka cluster. Just like the producer, the consumer uses of all servers in the cluster no matter which ones we list here.
The GROUP_ID_CONFIG identifies the consumer group of this consumer.
The KEY_DESERIALIZER_CLASS_CONFIG ("key.deserializer") is a Kafka Deserializer class for Kafka record keys that implements the Kafka Deserializer interface. Notice that we set this
to LongDeserializer as the message ids in our example are longs.
The VALUE_DESERIALIZER_CLASS_CONFIG ("value.deserializer") is a Kafka Serializer class for Kafka record values that implements the Kafka Deserializer interface. Notice that we set this
to StringDeserializer as the message body in our example are strings.
Important notice that you need to subscribe the consumer to the
topic consumer.subscribe(Collections.singletonList(TOPIC));. The subscribe method takes a list of topics to subscribe to, and this list will replace the current subscriptions if any.
Process messages from Kafka with Consumer
Now, let's process some records with our Kafka Producer.

Add the following code that will process the message from the topic;

```
static void runConsumer() throws InterruptedException {
        final Consumer<Long, String> consumer = createConsumer();

        final int giveUp = 100;
        int noRecordsCount = 0;

        while (true) {
            final ConsumerRecords<Long, String> consumerRecords = consumer.poll(1000);

            if (consumerRecords.count() == 0) {
                noRecordsCount++;
                if (noRecordsCount > giveUp)
                    break;
                else
                    continue;
            }

            consumerRecords.forEach(record -> {
                System.out.printf("Consumer Record:(%d, %s, %d, %d)\n", record.key(),
record.value(),
                        record.partition(), record.offset());
            });

            consumer.commitAsync();
        }
        consumer.close();
```

```
            System.out.println("DONE");
        }

}
```

Notice you use ConsumerRecords which is a group of records from a Kafka topic partition.
The ConsumerRecords class is a container that holds a list of ConsumerRecord(s) per partition for a particular topic. There is one ConsumerRecord list for every topic partition returned by a the consumer.poll().
Notice if you receive records (consumerRecords.count()!=0), then runConsumer method calls consumer.commitAsync() which commit offsets returned on the last call to consumer.poll(...) for all the subscribed list of topic partitions.
Kafka Consumer Poll method
The poll method returns fetched records based on current partition offset. The poll method is a blocking method waiting for specified time in seconds. If no records are available after the time period specified, the poll method returns an empty ConsumerRecords.
When new records become available, the poll method returns straight away.
You can control the maximum records returned by the poll() with props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 100);. The poll method is not thread safe and is not meant to get called from multiple threads.

Running the Kafka Consumer
Next you define the main method.

```
public class KafkaConsumerExample {

  public static void main(String... args) throws Exception {
     runConsumer();
  }
}
```

The main method just calls runConsumer.
Try running the consumer and producer.
Run the consumer from your IDE. Then run the producer from the last tutorial from your IDE. You should see the consumer get the records that the producer sent.

MyKafkaConsumer.java | MySimpleKafkaProducer.java

```java
    public static void main(String... args) throws Exception {
          runConsumer();
      }


    private static Consumer<Long, String> createConsumer() {
        final Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, BOOTSTRAP_SERVERS);
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "KafkaExampleConsumer");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, LongDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName

        // Create the consumer using props.
        final Consumer<Long, String> consumer = new KafkaConsumer<>(props);

        // Subscribe to the topic.
        consumer.subscribe(Collections.singletonList(TOPIC));
        return consumer;
    }
```

Problems | Target Platform State | Console

```
<terminated> MyKafkaConsumer [Java Application] D:\MyExperiment\jre\bin\javaw.exe (Jun 16, 2018, 5:20:28 PM)
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
DONE
Picked up _JAVA_OPTIONS: -Djava.net.preferIPv4Stack=true
```

As you can see there are no records. Why?? It was already consumed by the terminal windows which offset is committed.
Stop the consumer console.
Execute the Producer and then execute the consumer

```
Problems  Target Platform State  Console
<terminated> MyKafkaConsumer [Java Application] D:\MyExperiment\jre\bin\javaw.exe (Jun 16, 2018, 5:34:51 PM)
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Exception in thread "main" org.apache.kafka.common.errors.SerializationException: Error deserializing key/va
Caused by: org.apache.kafka.common.errors.SerializationException: Size of data received by LongDeserializer
Picked up _JAVA_OPTIONS: -Djava.net.preferIPv4Stack=true
Consumer Record:(1529150682676, Hello Kafka 1529150682676, 6, 2)
Consumer Record:(1529150682678, Hello Kafka 1529150682678, 3, 1)
Consumer Record:(1529150682677, Hello Kafka 1529150682677, 12, 1)
```

Logging set up for Kafka
If you don't set up logging well, it might be hard to see the consumer get the messages.
Kafka like most Java libs these days uses sl4j. You can use Kafka with Log4j, Logback or JDK logging. We used logback in our maven build (compile 'ch.qos.logback:logback-classic:1.2.2').

src/main/resources/logback.xml

```
<configuration>

    <appender name="STDOUT"
        class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
            </pattern>
        </encoder>
    </appender>
```

```
        <logger name="org.apache.kafka" level="INFO" />
        <logger name="org.apache.kafka.common.metrics" level="INFO" />

        <root level="debug">
                <appender-ref ref="STDOUT" />
        </root>
</configuration>
```

Notice that we set org.apache.kafka to INFO, otherwise we will get a lot of log messages. You should run it set to debug and read through the log messages. It gives you a flavor of what Kafka is doing under the covers. Leave org.apache.kafka.common.metrics or what Kafka is doing under the covers is drowned by metrics logging.

---------- ---------- ----------

Lab End Here

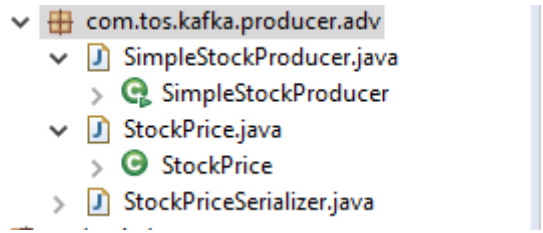
4. Java API – Producer – II

In this lab, we will be sending a JSON object using a custom serialize and in async mode.
The Stock Price Producer example has the following classes:

- StockPrice - holds a stock price has a name, dollar, and cents
- SimpleStockProducer - Configures and creates KafkaProducer, StockSender list, ThreadPool (ExecutorService), starts StockSender runnable into thread pool
- StockPriceSerializer - can serialize a StockPrice into byte[]

You can use the same maven project we have created in the producer example. In order to categorize the tutorials, let us create a package com.tos.kafka.aproducer, all the classes created for this lab will be stored inside this package.
Package Name:
com.tos.kafka.producer.adv

At the end of this lab we will have the following class.



Define all dependencies in the pom.xml as shown below:

```
<properties>
        <maven.compiler.target>1.8</maven.compiler.target>
        <maven.compiler.source>1.8</maven.compiler.source>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.apache.kafka</groupId>
            <artifactId>kafka-clients</artifactId>
            <version>1.1.0</version>
            <scope>compile</scope>
        </dependency>
        <!-- https://mvnrepository.com/artifact/io.advantageous.boon/boon-json -->
        <dependency>
            <groupId>io.advantageous.boon</groupId>
            <artifactId>boon-json</artifactId>
            <version>0.6.6</version>
        </dependency>

        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-ext</artifactId>
            <version>1.7.13</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
```

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.6.1</version>
    <configuration>
        <source>1.8</source>
        <target>1.8</target>
    </configuration>
</plugin>
<plugin>
    <artifactId>maven-assembly-plugin</artifactId>
    <configuration>
        <archive>
            <manifest>
                <mainClass>fully.qualified.MainClass</mainClass>
            </manifest>
        </archive>
        <descriptorRefs>
            <descriptorRef>jar-with-dependencies</descriptorRef>
        </descriptorRefs>
    </configuration>
    <executions>
        <execution>
            <id>make-assembly</id> <!-- this is used for inheritance merges -->
            <phase>package</phase> <!-- bind to the packaging phase -->
            <goals>
                <goal>single</goal>
            </goals>
        </execution>
```

```
                </executions>
            </plugin>
        </plugins>
    </build>
```

StockPrice

The StockPrice is a simple domain object that holds a stock price has a name, dollar, and cents. The StockPrice knows how to convert itself into a JSON string.

```
package com.tos.kafka.producer.adv;
import io.advantageous.boon.json.JsonFactory;

public class StockPrice {

      private final int dollars;
   private final int cents;
   private final String name;

   public StockPrice(final String json) {
      this(JsonFactory.fromJson(json, StockPrice.class));
   }


   public StockPrice() {
      dollars = 0;
      cents = 0;
      name = "";
   }

   public StockPrice(final String name, final int dollars, final int cents) {
      this.dollars = dollars;
      this.cents = cents;
      this.name = name;
   }
```

```
public StockPrice(final StockPrice stockPrice) {
   this.cents = stockPrice.cents;
   this.dollars = stockPrice.dollars;
   this.name = stockPrice.name;
}


public int getDollars() {
   return dollars;
}


public int getCents() {
   return cents;
}


public String getName() {
   return name;
}


@Override
public String toString() {
   return "StockPrice{" +
         "dollars=" + dollars +
         ", cents=" + cents +
         ", name='" + name + '\'' +
```

```
        '}';
}


@Override
public boolean equals(Object o) {
   if (this == o) return true;
   if (o == null || getClass() != o.getClass()) return false;

   StockPrice that = (StockPrice) o;

   if (dollars != that.dollars) return false;
   if (cents != that.cents) return false;
   return name != null ? name.equals(that.name) : that.name == null;
}

@Override
public int hashCode() {
   int result = dollars;
   result = 31 * result + cents;
   result = 31 * result + (name != null ? name.hashCode() : 0);
   return result;
}


public String toJson() {
   return "{" +
         "\"dollars\": " + dollars +
         ", \"cents\": " + cents +
```

```
            ", \"name\": \"" + name + '\"' +
            '}';
    }
}
```

Create a class SimpleStockProducer, which will be our main program that will send records to Broker. Here we will demonstrate the pushing of JSON document using serialize component. SimpleStockProducer import classes and sets up a logger. It has a main() method to create a KafkaProducer instance. It has a initProducer() method to initialize bootstrap servers, client id, key serializer and custom serializer (StockPriceSerializer). It has a main() method that creates the producer, creates a StockSender list passing each instance the producer, and then it runs each stockSender in its own thread.

```
public class SimpleStockProducer
```

Add the following import in the above class.

```
package com.tos.kafka.producer.adv;

import java.util.Properties;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.TimeUnit;

import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.Producer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.clients.producer.RecordMetadata;
```

Add the following method to initialize the properties of the Producer.

```
public final static String TOPIC = "stock-prices";

    public static Properties initProducer() {
        // Assign topicName to string variable

        // create instance for properties to access producer configs
        Properties props = new Properties();

        // Assign localhost id
        props.put("bootstrap.servers", "192.168.139.129:9093");
        // props.put("bootstrap.servers", "localhost:9092");

        // Set acknowledgements for producer requests.
        // props.put("acks", "all");
        props.put("enable.auto.commit", "true");
        // If the request fails, the producer can automatically retry,
        props.put("retries", 1);

        // Specify buffer size in config
        props.put("batch.size", 1);

        // Reduce the no of requests less than 0
        props.put("linger.ms", 100);

        // The buffer.memory controls the total amount of memory available to the
```

```
        // producer for buffering.
        props.put("buffer.memory", 302);

        props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");

        props.put("value.serializer", StockPriceSerializer.class.getName());
        // props.put("transactional.id", "my-transactional-id");
        return props;

    }
```

The main method that will create the producer and send message to Broker

```
    public static void main(String[] args) {
        Properties props = initProducer();
        Producer<String, StockPrice> producer = new KafkaProducer<String, StockPrice>(props);
        //sendMessage(producer, TOPIC);
        //sendMessageAsync(producer, TOPIC);

    }
```

Add the following function in the main class to send records synchronously to Kafka Producer.

```
public static void sendMessage(Producer<String, StockPrice> producer, String topic) {

        for (int i = 0; i < 2; i++) {
            try {
```

```
                ProducerRecord <String, StockPrice> record =  createRandomRecord(i);
                RecordMetadata future = producer.send(record).get();
                System.out.println(">>>" + future.topic() + " Offset:" + future.offset());
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            } catch (ExecutionException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
            producer.flush();
            System.out.println("Message sent successfully");
        }
        producer.close();
        // test(producer);
        System.out.println("Message sent successfully & Close");
    }
```

Method that will create Two Producer records using StockPrice class

```
    private static ProducerRecord<String, StockPrice> createRandomRecord(int i) {
        final int dollarAmount = i* 200;
        final int centAmount = i * 1000;
        final StockPrice stockPrice = new StockPrice(" STOCK"+ i, dollarAmount, centAmount);
        return new ProducerRecord<>(TOPIC, stockPrice.getName(), stockPrice);
    }
```

To send records asynchronously with Kafka Producer the following method needs to be added.

Kafka provides an asynchronous send method to send a record to a topic. Let's use this method to send some message ids and messages to the Kafka topic we created earlier. The big difference here will be that we use a lambda expression to define a callback.

```
// Sending Message Asynchronously
    public static void sendMessageAsync(Producer<String, StockPrice> producer, String topic) {
        final CountDownLatch countDownLatch = new CountDownLatch(2);
        for (int i = 0; i < 2; i++) {
            try {
                ProducerRecord <String, StockPrice> record =  createRandomAsyncRecord(i);

                long time = System.currentTimeMillis();
                  // Register a call back.
                        producer.send(record, (metadata, exception) -> {
                    long elapsedTime = System.currentTimeMillis() - time;
                    if (metadata != null) {
                       System.out.printf("sent record(key=%s value=%s) " +
                                "meta(partition=%d, offset=%d) time=%d\n",
                            record.key(), record.value(), metadata.partition(),
                            metadata.offset(), elapsedTime);
                    } else {
                       exception.printStackTrace();
                    }
                    countDownLatch.countDown();
                  });

                     countDownLatch.await(25, TimeUnit.SECONDS);

            } catch (InterruptedException e) {
```

```
                // TODO Auto-generated catch block
                e.printStackTrace();
            } catch (Exception e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
            producer.flush();
            System.out.println("Message sent successfully");
        }
        producer.close();
        // test(producer);
        System.out.println("Message sent successfully & Close");
    }
```

Notice the use of a CountDownLatch so we can send all N messages and then wait for them all to send.

Async Interface Callback and Async Send Method

Kafka defines a Callback interface that you use for asynchronous operations. The callback interface allows code to execute when the request is complete. The callback executes in a background I/O thread so it should be fast (don't block it). The onCompletion(RecordMetadata metadata, Exception exception) gets called when the asynchronous operation completes. The metadata gets set (not null) if the operation was a success, and the exception gets set (not null) if the operation had an error.

The async send method is used to send a record to a topic, and the provided callback gets called when the send is acknowledged. The send method is asynchronous, and when called returns immediately once the record gets stored in the buffer of records waiting to post to the Kafka broker. The send method allows sending many records in parallel without blocking to wait for the response after each one.

Since the send call is asynchronous it returns a Future for the RecordMetadata that will be assigned to this record. Invoking get() on this future will block until the associated request completes and then return the metadata for the record or throw any exception that occurred while sending the record. KafkaProducer

In the above code, the following line overrides the onCompletion(RecordMetadata metadata, Exception exception) method.

```
producer.send(record, (metadata, exception)
```

--- --- --- ---

Add the following code that will create Record for sending to broker.

```
private static ProducerRecord<String, StockPrice> createRandomAsyncRecord(int i) {
        final int dollarAmount = i* 200;
        final int centAmount = i * 1000;
        final StockPrice stockPrice = new StockPrice(" Async STOCK "+ i, dollarAmount,
centAmount);
        return new ProducerRecord<>(TOPIC, stockPrice.getName(), stockPrice);
    }
```

Custom Serializers

We are not using built-in Kafka serializers. We are writing your own custom serializer. You just need to be able to convert your custom keys and values using the serializer convert to and convert from byte arrays (byte[]). Serializers work for keys and values, and you set them up with the Kafka Producer properties value.serializer, and key.serializer. The StockPriceSerializer will serialize StockPrice into bytes.

Create the following Custom Serializer class

```
package com.tos.kafka.producer.adv;


import java.nio.charset.StandardCharsets;
import java.util.Map;

import org.apache.kafka.common.serialization.Serializer;



public class StockPriceSerializer implements Serializer<StockPrice> {

   @Override
   public byte[] serialize(String topic, StockPrice data) {
      return data.toJson().getBytes(StandardCharsets.UTF_8);
   }

   @Override
   public void configure(Map<String, ?> configs, boolean isKey) {
   }

   @Override
   public void close() {
   }
}
```

You need to override the serialize method to convert JSON to bytes so that it can stream the stock price json document to broker.
Let us test the application now.

Start the broker and create the topic if not created ()

#sh start3Brokers.sh

```
[root@tos scripts]# sh start3Brokers.sh
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
Start zookeeper & 3 Brokers Successfully
[root@tos scripts]# jps
3025 QuorumPeerMain
3041 Kafka
3042 Kafka
3043 Kafka
3854 Jps
[root@tos scripts]# jps
```

/opt/kafka/bin/kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1 --topic stock-prices

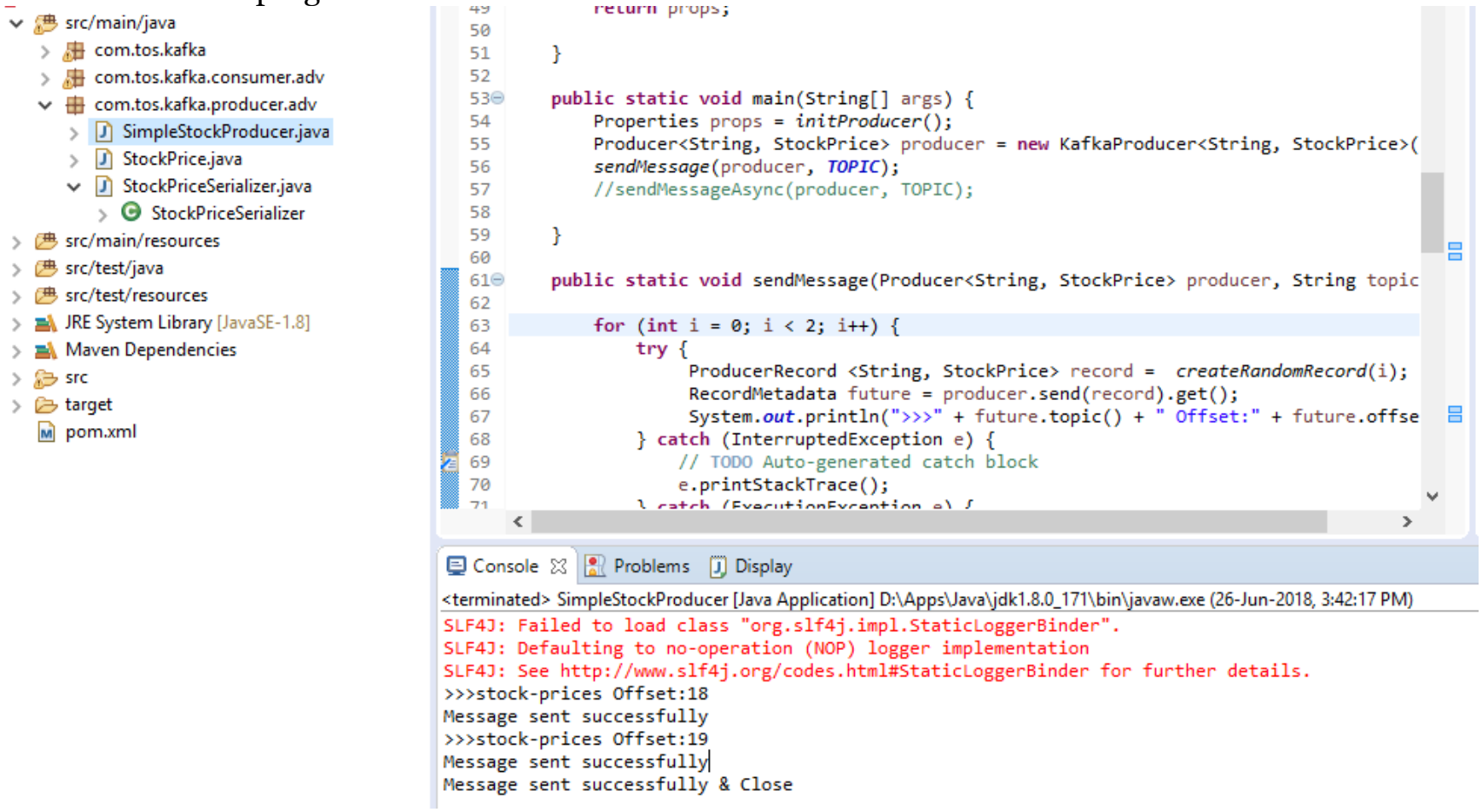
We will send the JSON message synchronously.

In the main method ensure that you have uncommented *sendMessage() and commented sendMessageAsync()*

```
public static void main(String[] args) {
    Properties props = initProducer();
    Producer<String, StockPrice> producer = new KafkaProducer<String, StockPrice>(
    sendMessage(producer, TOPIC);
    //sendMessageAsync(producer, TOPIC);

}
```

Open a terminal and execute the following command to consume the message:
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic stock-prices

Execute the main program.



The offset of the message will be printed in the console of the producer. (Ensure that kafka server IP and port are mention accordingly to your setting)
You should be able to view 2 json message in the consumer console.

```
[root@tos scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
tos.master.com:9093 --topic stock-prices
{"dollars": 0, "cents": 0, "name": " STOCK0"}
{"dollars": 200, "cents": 1000, "name": " STOCK1"}
```

Let us execute the send message in ascync mode. For this comment the sync method and uncomment async as shown below.

```
    public static void main(String[] args) {
        Properties props = initProducer();
        Producer<String, StockPrice> producer = new KafkaProducer<String, StockPrice>
        //sendMessage(producer, TOPIC);
        sendMessageAsync(producer, TOPIC);

    }
```

Execute the main program now. After a couple of seconds you should have console as shown below:

```
<terminated> SimpleStockProducer [Java Application] D:\Apps\Java\jdk1.8.0_171\bin\javaw.exe (26-Jun-2018, 3:44:57 PM)
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
sent record(key= Async STOCK 0 value=StockPrice{dollars=0, cents=0, name=' Async STOCK 0'}) meta(partition=0, offset=20) time=261
Message sent successfully
sent record(key= Async STOCK 1 value=StockPrice{dollars=200, cents=1000, name=' Async STOCK 1'}) meta(partition=0, offset=21) time=105
Message sent successfully
Message sent successfully & Close
```

We have printed the meta data of the record in the call back method. Here it mentions the partition also. You can verify from the consume console also. Here you can see two asnyc stock being consume from the topic, stock-prices.

```
[root@tos scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
tos.master.com:9093 --topic stock-prices
{"dollars": 0, "cents": 0, "name": " STOCK0"}
{"dollars": 200, "cents": 1000, "name": " STOCK1"}
{"dollars": 0, "cents": 0, "name": " Async STOCK 0"}
{"dollars": 200, "cents": 1000, "name": " Async STOCK 1"}
```

Let us verify the Partitioning of message too.
Create a topic with 4 partitions as shown below:
/opt/kafka/bin/kafka-topics.sh --create --zookeeper tos.master.com:2181 --replication-factor 2 --partitions 4 --topic mystock-prices

```
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --create --zookeeper tos.mast
er.com:2181 --replication-factor 2 --partitions 4 --topic mystock-prices
Created topic "mystock-prices".
```

Update the topic name as shown below.

```
14
15 public class SimpleStockProducer {
16
17     public final static String TOPIC = "mystock-prices";
18
19     public static Properties initProducer() {
```

We have created the topi, mystock-prices with 4 partitions so that message will be distributed across the partition.

Ensure that the following method is not commented.

```
public static void main(String[] args) {
    Properties props = initProducer();
    Producer<String, StockPrice> producer = new KafkaProducer<String, StockPrice
    //sendMessage(producer, TOPIC);
    sendMessageAsync(producer, TOPIC);

}
```

We will send 4 messages as shown, updated the counter to 4.

```
// Sending Message Asynchronously
public static void sendMessageAsync(Producer<String, StockPrice> producer, String
    final CountDownLatch countDownLatch = new CountDownLatch(2);
    for (int i = 0; i < 4; i++) {
        try {
            ProducerRecord <String, StockPrice> record =  createRandomAsyncRecord

                long time = System.currentTimeMillis();
                    // Register a call back.
                    producer.send(record, (metadata, exception) -> {
                            long elapsedTime = System.currentTimeMillis() - time;
                            if (metadata != null) {
                                System.out.printf("sent record(key=%s value=%s) "
                                                "meta(partition=%d, offset=%d) tim
                                        record.key(), record.value(), metadata.par
                                        metadata.offset(), elapsedTime);
                            } else {
                                exception.printStackTrace();
                            }
```

Execute the main program.

As you can see from the console, there are distributed across different partition.

```
<terminated> SimpleStockProducer [Java Application] D:\Apps\Java\jdk1.8.0_171\bin\javaw.exe (26-Jun-2018, 4:58:35 PM)
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
sent record(key= Async STOCK 0 value=StockPrice{dollars=0, cents=0, name=' Async STOCK 0'}) meta(partition=2, offset=0) time=340
Message sent successfully
sent record(key= Async STOCK 1 value=StockPrice{dollars=200, cents=1000, name=' Async STOCK 1'}) meta(partition=3, offset=0) time=158
Message sent successfully
sent record(key= Async STOCK 2 value=StockPrice{dollars=400, cents=2000, name=' Async STOCK 2'}) meta(partition=2, offset=1) time=10
Message sent successfully
sent record(key= Async STOCK 3 value=StockPrice{dollars=600, cents=3000, name=' Async STOCK 3'}) meta(partition=1, offset=0) time=20
Message sent successfully
Message sent successfully & Close
```

You can describe the partition as shown below:

/opt/kafka/bin/kafka-topics.sh --describe --zookeeper tos.master.com:2181 --topic mystock-prices

```
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --describe --zookeeper tos.master.com:2181  --topic mystock-pric
es
Topic:mystock-prices    PartitionCount:4        ReplicationFactor:2     Configs:
        Topic: mystock-prices   Partition: 0    Leader: 0       Replicas: 0,1   Isr: 0,1
        Topic: mystock-prices   Partition: 1    Leader: 1       Replicas: 1,2   Isr: 1,2
        Topic: mystock-prices   Partition: 2    Leader: 2       Replicas: 2,0   Isr: 2,0
        Topic: mystock-prices   Partition: 3    Leader: 0       Replicas: 0,2   Isr: 0,2
[root@tos scripts]#
```

Consumer:

#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server tos.master.com:9093 --topic mystock-prices

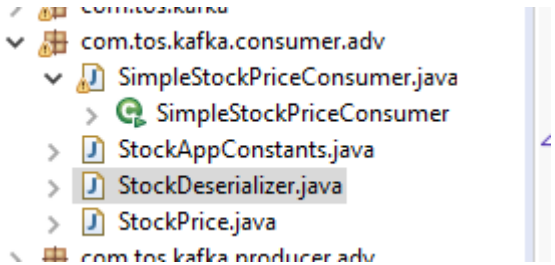
```
[root@tos scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
tos.master.com:9093 --topic mystock-prices
{"dollars": 0, "cents": 0, "name": " Async STOCK 0"}
{"dollars": 200, "cents": 1000, "name": " Async STOCK 1"}
{"dollars": 400, "cents": 2000, "name": " Async STOCK 2"}
{"dollars": 600, "cents": 3000, "name": " Async STOCK 3"}
{"dollars": 0, "cents": 0, "name": " Async STOCK 0"}
{"dollars": 200, "cents": 1000, "name": " Async STOCK 1"}
{"dollars": 400, "cents": 2000, "name": " Async STOCK 2"}
{"dollars": 600, "cents": 3000, "name": " Async STOCK 3"}
{"dollars": 800, "cents": 4000, "name": " Async STOCK 4"}
{"dollars": 1000, "cents": 5000, "name": " Async STOCK 5"}
```

Lab Ends here

5. Java API – Consumer – II

We will use the same Maven project that was created earlier. Let us create a separate package for storing all the classes that will be created in this tutorial.
Package : com.tos.kafka.consumer.adv

We will demonstrate how to use JSON deserialization and retry option.
At the end of the lab you should have the project structure as shown below:



Update the pom.xml with the following dependencies.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.tos.kafka</groupId>
  <artifactId>my-kafka-producer</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <dependencies>
          <dependency>
               <groupId>org.apache.kafka</groupId>
               <artifactId>kafka-clients</artifactId>
               <version>2.6.0</version>
          </dependency>
          <dependency>
               <groupId>org.apache.kafka</groupId>
               <artifactId>kafka-streams</artifactId>
               <version>2.6.0</version>
          </dependency>
          <dependency>
               <groupId>org.apache.kafka</groupId>
               <artifactId>kafka-streams-scala_2.13</artifactId>
               <version>2.6.0</version>
          </dependency>
   <dependency>
               <groupId>io.advantageous.boon</groupId>
               <artifactId>boon-json</artifactId>
               <version>0.6.6</version>
          </dependency>
          <dependency>
       <groupId>com.google.code.gson</groupId>
```

```
        <artifactId>gson</artifactId>
        <version>2.6.2</version>
    </dependency>

            <dependency>
                <groupId>org.slf4j</groupId>
                <artifactId>slf4j-ext</artifactId>
                <version>1.7.13</version>
            </dependency>

            </dependencies>
    <build>
            <plugins>
                <plugin>
                    <groupId>org.apache.maven.plugins</groupId>
                    <artifactId>maven-compiler-plugin</artifactId>
                    <version>3.6.1</version>
                    <configuration>
                        <source>1.8</source>
                        <target>1.8</target>
                    </configuration>
                </plugin>

            </plugins>


</build>


</project>
```

The Stock Price Consumer example has the following classes:

- StockPrice - holds a stock price has a name, dollar, and cents
- SimpleStockPriceConsumer - consumes StockPrices and display batch lengths for poll
- StockAppConstants - holds topic and broker list
- StockDeserializer - can deserialize a StockPrice from byte[]

StockDeserializer

The StockDeserializer just calls the JSON parser to parse JSON in bytes to a StockPrice object. Create the following class.

```
package com.tos.kafka.consumer.adv;

import org.apache.kafka.common.serialization.Deserializer;

import java.nio.charset.StandardCharsets;
import java.util.Map;

public class StockDeserializer implements Deserializer<StockPrice> {

  @Override
  public StockPrice deserialize(final String topic, final byte[] data) {
    return new StockPrice(new String(data, StandardCharsets.UTF_8));
  }
```

```
    @Override
    public void configure(Map<String, ?> configs, boolean isKey) {
    }

    @Override
    public void close() {
    }
}
```

We just need to override the deserialize() that convert the JSON bytes to Stock Price.

Create the following class which will be our data or domain value object.

```
package com.tos.kafka.consumer.adv;


import com.google.gson.Gson;

public class StockPrice {

   private final int dollars ;
    private final int cents ;
    private final String name ;
    static Gson gson = new Gson();

    public StockPrice(final String json) {
          this(gson.fromJson(json, StockPrice.class));
    }


    public StockPrice() {
        dollars = 0;
        cents = 0;
        name = "";
    }

    public StockPrice(final String name, final int dollars, final int cents) {
        this.dollars = dollars;
        this.cents = cents;
        this.name = name;
    }


    public StockPrice(final StockPrice stockPrice) {
```

```
        this.cents = stockPrice.cents;
        this.dollars = stockPrice.dollars;
        this.name = stockPrice.name;
    }


    public int getDollars() {
        return dollars;
    }


    public int getCents() {
        return cents;
    }


    public String getName() {
        return name;
    }


    @Override
    public String toString() {
        return "StockPrice{" +
                "dollars=" + dollars +
                ", cents=" + cents +
                ", name='" + name + '\'' +
                '}';
    }


    @Override
    public boolean equals(Object o) {
```

```
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;

        StockPrice that = (StockPrice) o;

        if (dollars != that.dollars) return false;
        if (cents != that.cents) return false;
        return name != null ? name.equals(that.name) : that.name == null;
    }

    @Override
    public int hashCode() {
        int result = dollars;
        result = 31 * result + cents;
        result = 31 * result + (name != null ? name.hashCode() : 0);
        return result;
    }


    public String toJson() {
        return "{" +
                "\"dollars\": " + dollars +
                ", \"cents\": " + cents +
                ", \"name\": \"" + name + '\"' +
                '}';
    }
}
```

The application constants that define the servers that we will be connecting by our consumer. Ensure that you change your system broker IP accordingly. Update the code with that of the following.

```
package com.tos.kafka.consumer.adv;

public class StockAppConstants {
        public final static String TOPIC = "stock-prices";
          public final static String BOOTSTRAP_SERVERS =
                "localhost:9092";

}
```

SimpleStockPriceConsumer uses createConsumer method to create a KafkaConsumer instance, subscribes to stock-prices topics and has a custom deserializer.
It has a runConsumer() method that drains topic, creates List of current stocks and calls displayRecordsStatsAndStocks() method.
The method displayRecordsStatsAndStocks() prints out size of each partition read and total record count and prints out each stock at its current price.

Create a class SimpleStockPriceConsumer and import the following class.

```
package com.tos.kafka.consumer.adv;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Properties;
import java.util.UUID;

import org.apache.kafka.clients.consumer.Consumer;
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.serialization.StringDeserializer;
```

Add the following method that will create KafkaConsumer.

```
private static Consumer<String, StockPrice> createConsumer() {
		final Properties props = new Properties();
		props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
StockAppConstants.BOOTSTRAP_SERVERS);
		// props.put(ConsumerConfig.GROUP_ID_CONFIG, "KafkaExampleAdvConsumer");
		props.put(ConsumerConfig.GROUP_ID_CONFIG, "KA" +
UUID.randomUUID().toString());
		// props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
		// "KafkaExampleAdvConsumer");
		props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
		props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StockDeserializer.class.getName());
		props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500);
		// Create the consumer using props.
		final Consumer<String, StockPrice> consumer = new KafkaConsumer<>(props);
		// Subscribe to the topic.
		consumer.subscribe(Collections.singletonList(StockAppConstants.TOPIC));
		return consumer;
	}
```

Add the following method that will call the poll method that will pull the message from the broker

```
static void runConsumer() throws InterruptedException {
		final Consumer<String, StockPrice> consumer = createConsumer();
		final List<StockPrice> map = new ArrayList<StockPrice>();
		try {
```

```
            final int giveUp = 1;
            int noRecordsCount = 0;
            while (true) {
                consumer.seekToBeginning(consumer.assignment());
                ConsumerRecords<String, StockPrice> consumerRecords = consumer.poll(100);
                if (consumerRecords.count() == 0) {
                    noRecordsCount++;
                    if (noRecordsCount > giveUp)
                        break;
                    else
                        continue;
                }
                consumerRecords.forEach(record -> {
                    map.add(record.value());
                });
                displayRecordsStatsAndStocks(map, consumerRecords);
                consumer.commitAsync();
            }
        } finally {
            consumer.close();
        }
        System.out.println("DONE");
    }
```

The following method will print the records fetch from the Topic.

```
private static void displayRecordsStatsAndStocks(final List<StockPrice> stockPriceMap,
        final ConsumerRecords<String, StockPrice> consumerRecords) {
    System.out.printf("New ConsumerRecords partition count: %d Message count: %d\n",
            consumerRecords.partitions().size(), consumerRecords.count());
    stockPriceMap.forEach((stockPrice) -> System.out.printf("ticker %s price %d.%d \n",
stockPrice.getName(),
            stockPrice.getDollars(), stockPrice.getCents()));
    System.out.println();
}
```

Finally update the following method to call the method that consume the record.

```
public static void main(String[] args) {
    try {
        runConsumer();
    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }

}
```

You can execute the main method now. Ensure that you have started the broker before going ahead.
#start3Brokers.sh

```
[root@tos scripts]#
[root@tos scripts]# sh start3Brokers.sh
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
Start zookeeper & 3 Brokers Successfully
[root@tos scripts]# jps
2675 Kafka
2676 Kafka
2661 QuorumPeerMain
3429 -- process information unavailable
3386 Jps
[root@tos scripts]#
```

Execute the main program

```
Console   Problems   Display
<terminated> SimpleStockPriceConsumer [Java Application] D:\Apps\Java\jdk1.8.0_171\bin\javaw.exe (27-Jun-2018, 12:53:17 PM)
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
New ConsumerRecords partition count: 1 Message count: 22
ticker  STOCK0 price 0.0
ticker  STOCK1 price 200.1000
ticker  STOCK0 price 0.0
ticker  STOCK1 price 200.1000
ticker  STOCK0 price 0.0
ticker  STOCK1 price 200.1000
ticker  STOCK0 price 0.0
ticker  STOCK1 price 200.1000
ticker  STOCK0 price 0.0
```

As you can see above, it printed the number of partition count and the number of messages. It may vary depending on the number of messages that was pushed by your producer.

---------- ----------Lab Ends here ----------

```
58
59                  wordCountMap.forEach((key, value) -> producer.send(new ProducerRecord<String, String>(OU
60
61                  Map<TopicPartition, OffsetAndMetadata> offsetsToCommit = new HashMap<>();
62
63                  for (TopicPartition partition : records.partitions()) {
64                      List<ConsumerRecord<String, String>> partitionedRecords = records.records(partition)
65
66                      long offset = partitionedRecords.get(partitionedRecords.size() - 1).offset();
67                      System.out.println(">>>" + offset + " - " + partition.partition());
68                      offsetsToCommit.put(partition, new OffsetAndMetadata(offset + 1));
69                  }
70
71                  boolean flag = true;
72
73                  producer.sendOffsetsToTransaction(offsetsToCommit, CONSUMER_GROUP_ID);
74
75                  if (flag == true) {
76                      throw new Exception();
77                  }
78
79                  producer.commitTransaction();
```

Stop the program and start it again

And type a sentence on the producer terminal:

```
(base) Henrys-MacBook-Air:~ henrypotsangbam$ kafka-console-producer --broker-list localhost:9092 --topic input
>Let us count the number of count
>transaction works
```

Verify the console on the consumer and the execution of the program.

No changes in the console.

```
control-center is [UP]
(base) Henrys-MacBook-Air:~ henrypotsangbam$ kafka-console-consumer  --bootstrap-server localhost:9092 --topi
c output --from-beginning
1
1
1
2
1
1
1
1


1
1
1
1
```

The program stop execution:

```
Problems  Javadoc  Declaration  Console  Progress
<terminated> TransactionalWordCount [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_171.jdk/C[redacted]Home/bin/java (20 Aug, 2020 10:26:23 PM – 1
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
>>>2 – 0
```

changes the flag to false and execute the program now.

```
control-center is [UP]
(base) Henrys-MacBook-Air:~ henrypotsangbam$ kafka-console-consumer  --bootstrap-server localhost:9092 --topi
c output --from-beginning
1
1
1
2
1
1
1
1


1
1
1
1
1
1
1
```

Additional Two records have been consumed again

As observe because of the transaction, the messages are sent to the output topic once only.

---------- Lab Ends Here ----------